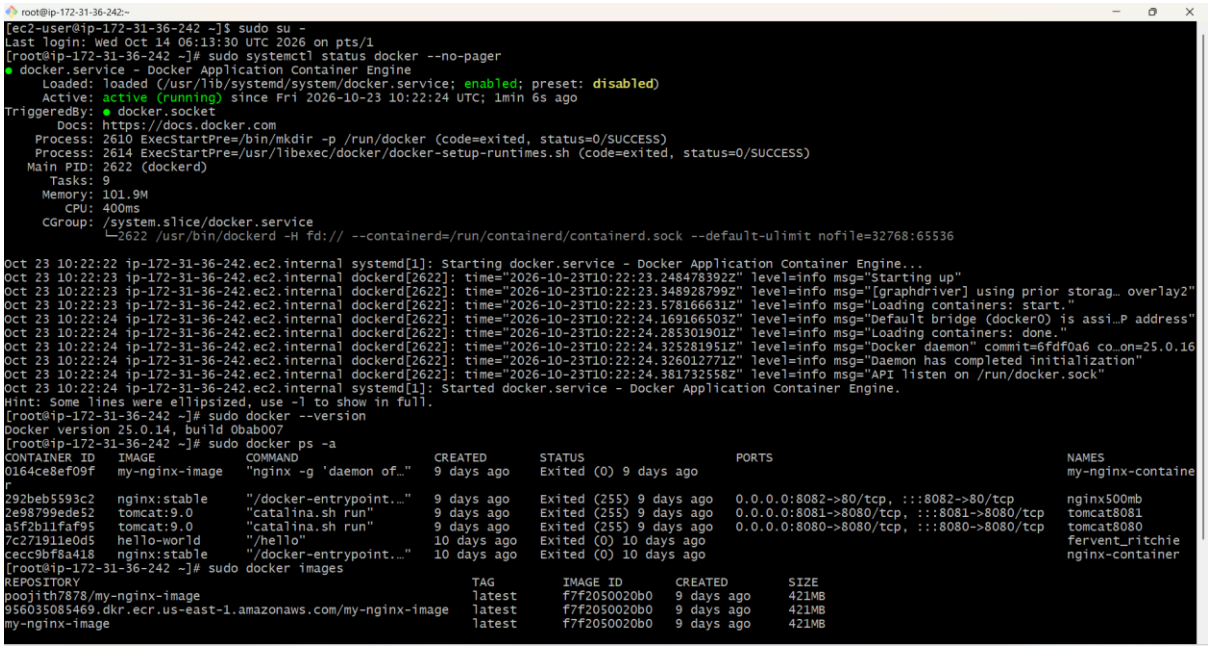
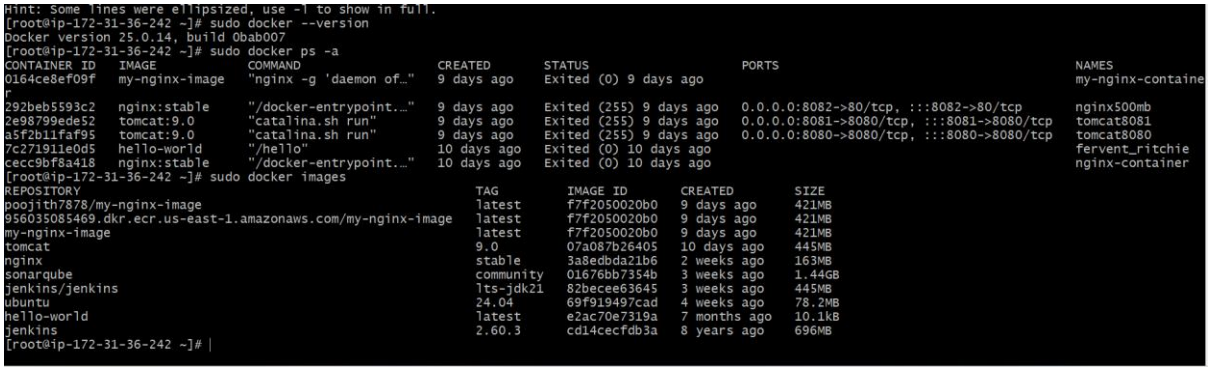
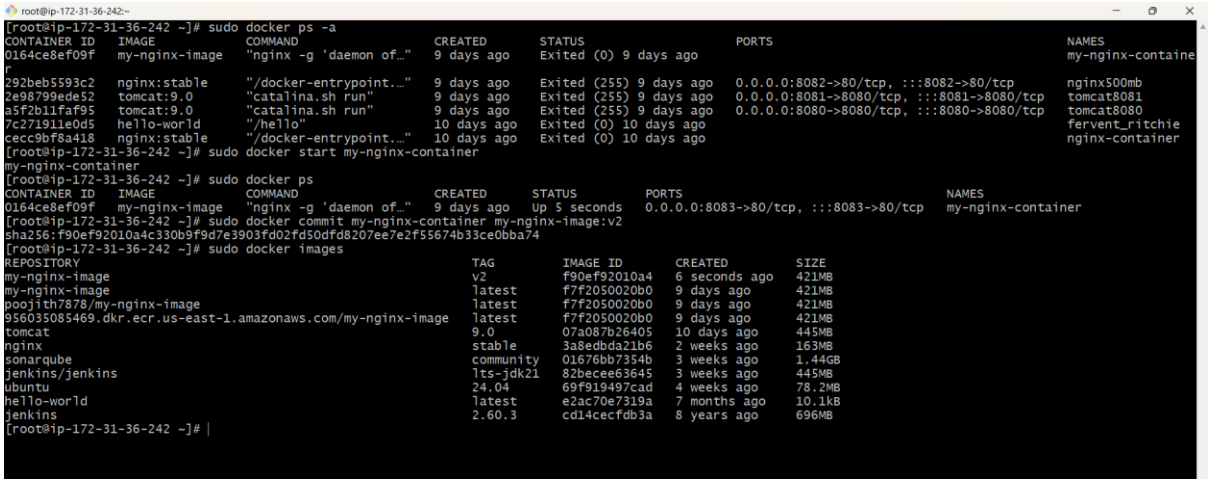
1. Create a image from running container.

Implementation steps

- Checked the existing Docker containers using → docker ps -a.
- Started the existing container using → docker start my-nginx-container.
- Verified the container was running using → docker ps.
- Created a new image from the running container using
  docker commit my-nginx-container my-nginx-image:v2.
- Verified the newly created image using → docker images.







2. Copy image from local machine to docker server and load the image.

Implementation Steps

Step 1: Verify the Docker image on the source EC2 instance.

sudo docker images

Identify the image my-nginx-image:v2.

Step 2: Save the Docker image as a TAR archive.

sudo docker save -o my-nginx-image-v2.tar my-nginx-image:v2

Step 3: Verify the archive.

ls -lh my-nginx-image-v2.tar

Step 4: Transfer the archive to the destination EC2 instance.

Copy the TAR file to the destination instance using SCP from a system that has access to the archive:

scp -i /path/to/key.pem my-nginx-image-v2.tar ec2-user@DESTINATION_PUBLIC_IP:/home/ec2-user/

Replace the key path and destination public IP with the actual values.

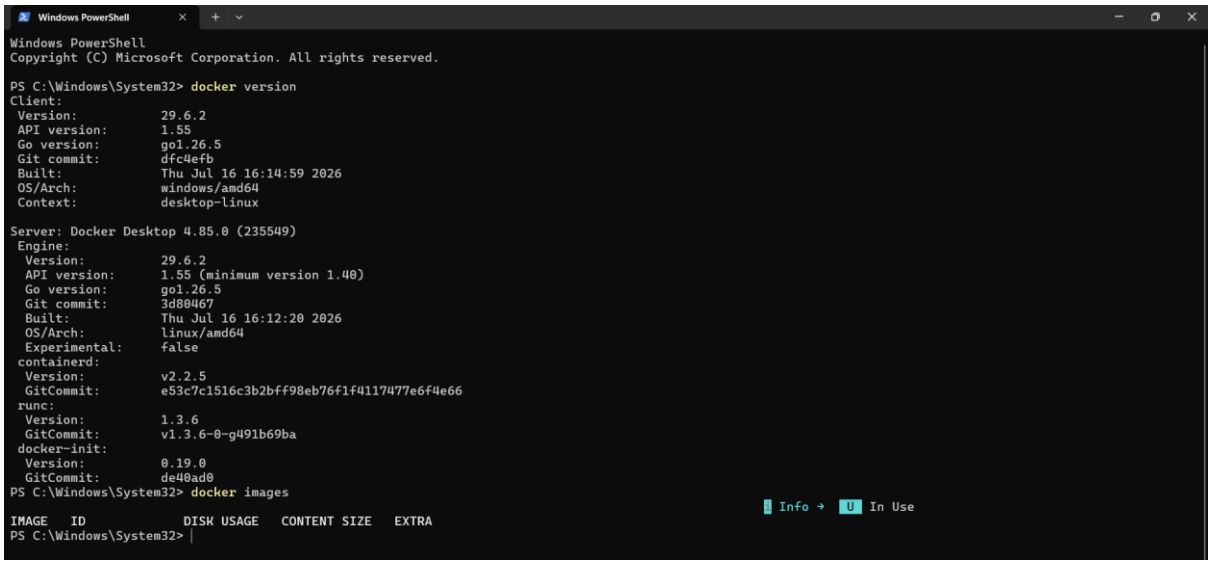
Step 5: Load the Docker image on the destination instance.

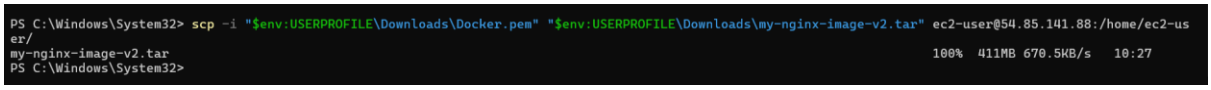
sudo docker load -i /home/ec2-user/my-nginx-image-v2.tar

Step 6: Verify the loaded image.

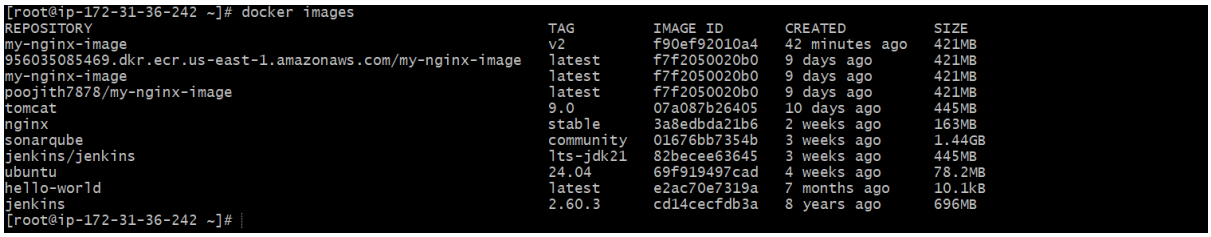
sudo docker images

Confirm that my-nginx-image:v2 appears in the image list.









3. Create Docker image using alpine and customize with tomcat.

Implementation steps

Step 1: Create a project directory.

mkdir alpine-tomcat

cd alpine-tomcat

Step 2: Create the Dockerfile.

vi Dockerfile

Add the following instructions:

```
FROM alpine:latest

RUN apk add --no-cache openjdk17-jre wget tar

ENV CATALINA_HOME=/opt/tomcat
ENV PATH=$CATALINA_HOME/bin:$PATH

WORKDIR /opt/tomcat

RUN wget -O /tmp/tomcat.tar.gz https://archive.apache.org/dist/tomcat/tomcat-9/v9.0.108/bin/apache-tomcat-9.0.108.tar.gz \
  && tar -xzf /tmp/tomcat.tar.gz --strip-components=1 \
  && rm /tmp/tomcat.tar.gz

EXPOSE 8080

CMD ["catalina.sh", "run"]
```

Save and exit the file.

Step 3: Build the Docker image.

```
sudo docker build -t alpine-tomcat:latest .
```

Step 4: Create and start the container.

```
sudo docker run -d --name alpine-tomcat-container -p 8084:8080 alpine-tomcat:latest
```

Step 5: Verify the running container.

```
sudo docker ps
```

Step 6: Check the Tomcat logs.

```
sudo docker logs alpine-tomcat-container
```

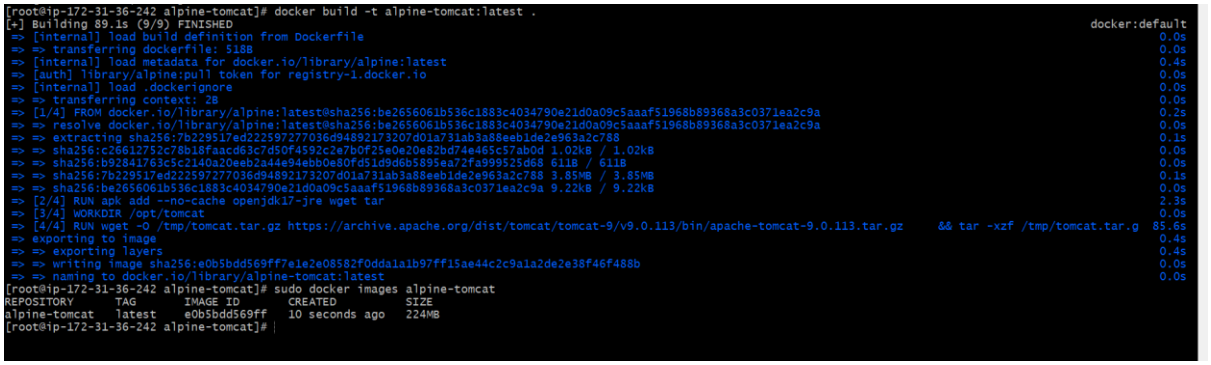
Step 7: Access Tomcat through a browser.

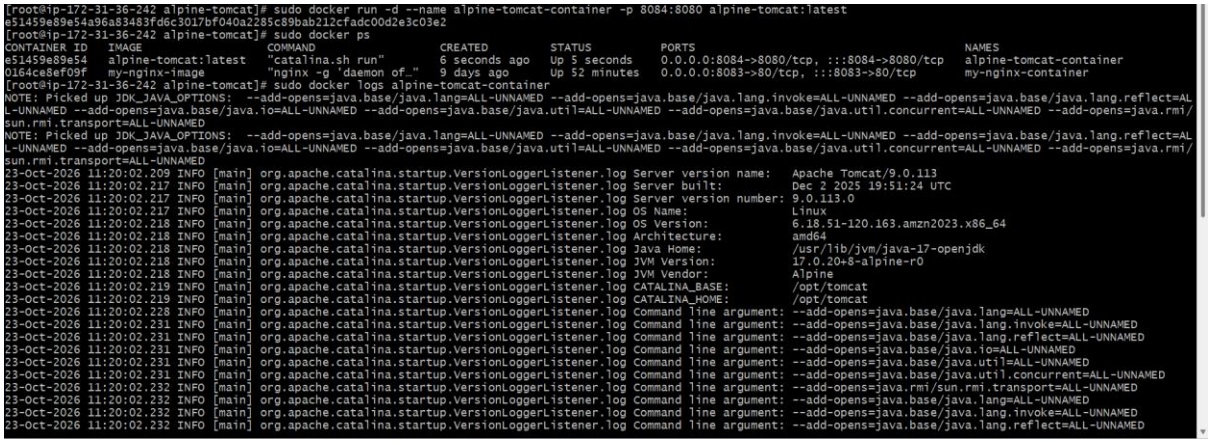
Open the following URL, replacing the placeholder with the EC2 public IP address:

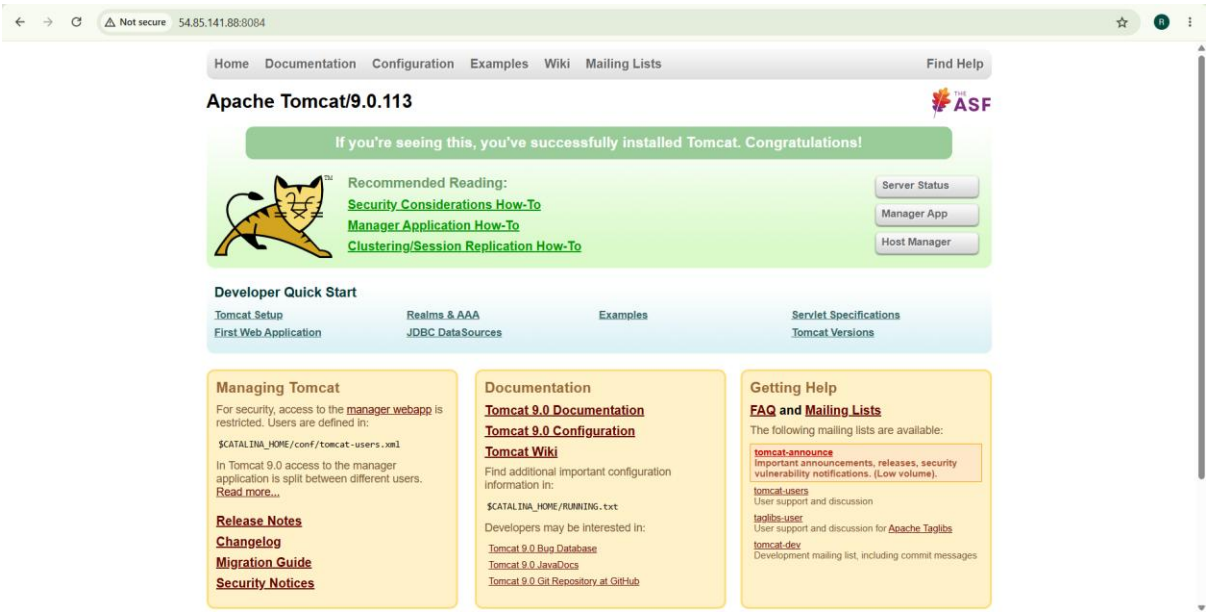
http://EC2_PUBLIC_IP:8084

Ensure the EC2 security group permits inbound TCP traffic on port 8084 from your IP address.









4. **Create single stage and multi stage docker file using this source code: https://github.com/betawins/multi-stage-example.git**

Implementation steps

Step 1: Install Git.

sudo yum install git -y

Step 2: Create a project directory.

mkdir multi-stage-lab

cd multi-stage-lab

Step 3: Clone the application repository.

git clone https://github.com/betawins/multi-stage-example.git .

Step 4: Inspect the project files.

ls -la

Review the available Dockerfiles and application source code.

Step 5: Build the single-stage image.

Use the single-stage Dockerfile supplied with the project. For example, if it is named Dockerfile.single:

sudo docker build -f Dockerfile.single -t multi-stage-example:single .

Step 6: Build the multi-stage image.

Build the image using the multi-stage Dockerfile supplied with the project:

sudo docker build -t multi-stage-example:v1 .

Step 7: Compare the image sizes.

sudo docker images

Record the sizes displayed for both images.

The completed practical recorded approximate image sizes of:

Image	Approximate size
Single-stage image	598 MB
Multi-stage image	172 MB

Step 8: Run the multi-stage container.

sudo docker run -d --name multi-stage-app -p 8085:8080 multi-stage-example:v1

Step 9: Verify the container.

sudo docker ps

Step 10: Access the application.

http://EC2_PUBLIC_IP:8085

Ensure the EC2 security group permits inbound TCP traffic on port 8085.



```
[root@ip-172-31-36-242 multi-stage-example]# vi Dockerfile
[root@ip-172-31-36-242 multi-stage-example]# vi Dockerfile
[root@ip-172-31-36-242 multi-stage-example]# cat Dockerfile
FROM maven:3.6.3-openjdk-8 AS builder

WORKDIR /app/source

COPY pom.xml .
COPY src ./src

RUN mvn clean package

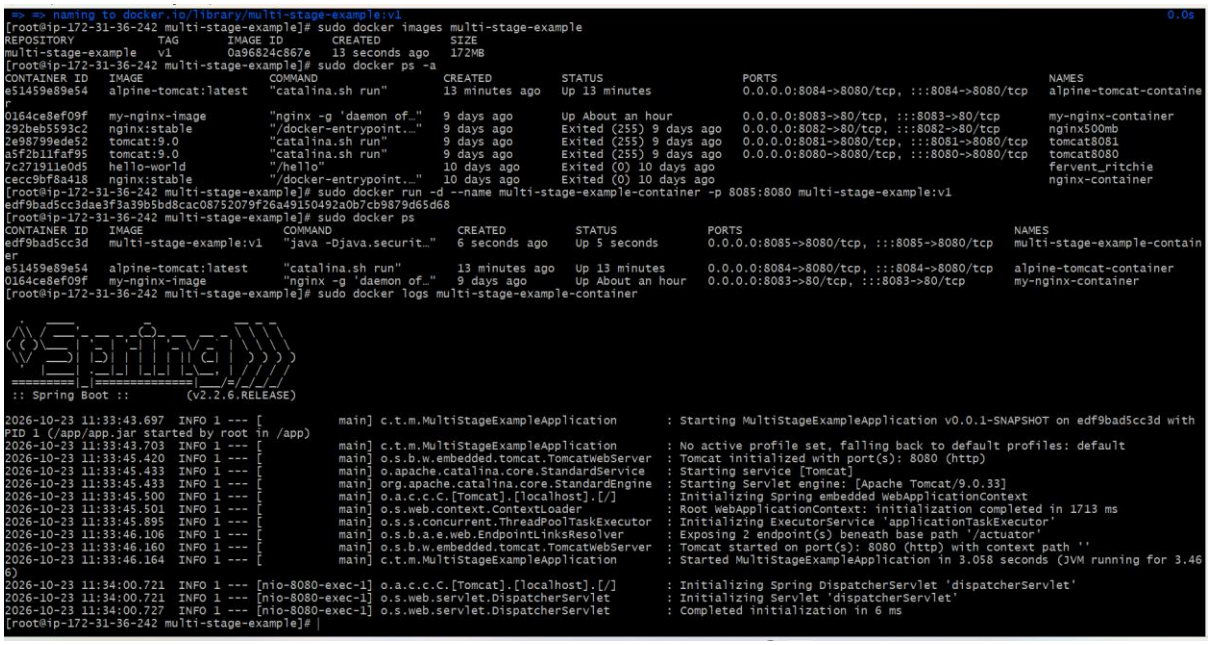
FROM eclipse-temurin:8-jre-alpine

WORKDIR /app

COPY --from=builder /app/source/target/*.jar /app/app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "/app/app.jar"]
[root@ip-172-31-36-242 multi-stage-example]# sudo docker build -t multi-stage-example:v1 .
[+] Building 40.9s (15/15) FINISHED                                                
```

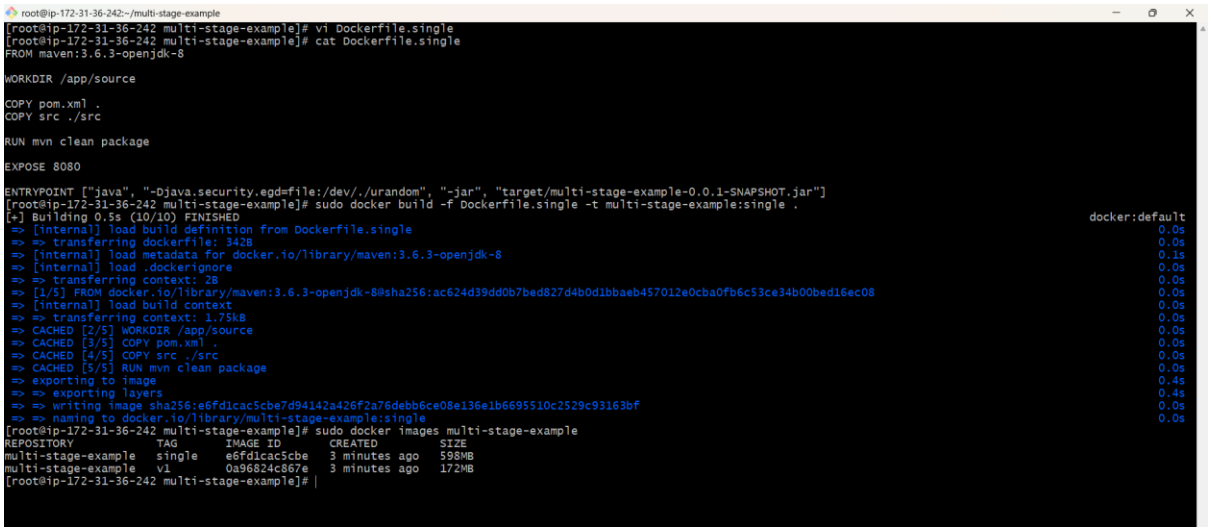




Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Fri Oct 23 11:34:00 GMT 2026
There was an unexpected error (type=Not Found, status=404).
No message available



5. Install docker compose and execute sample application.

Implementation steps

Step 1: Verify Docker Compose.

docker compose version

Step 2: Create a project directory.

mkdir nginx-compose

cd nginx-compose

Step 3: Create the Compose file.

vi compose.yaml

Add the following configuration:

```
services:
  nginx:
    image: nginx:stable
    container_name: nginx-compose-container
    ports:
      - "8086:80"
    restart: unless-stopped
```

Save and exit the file.

Step 4: Start the application.

sudo docker compose up -d

Step 5: Verify the Compose service.

sudo docker compose ps

Step 6: View the application logs.

sudo docker compose logs

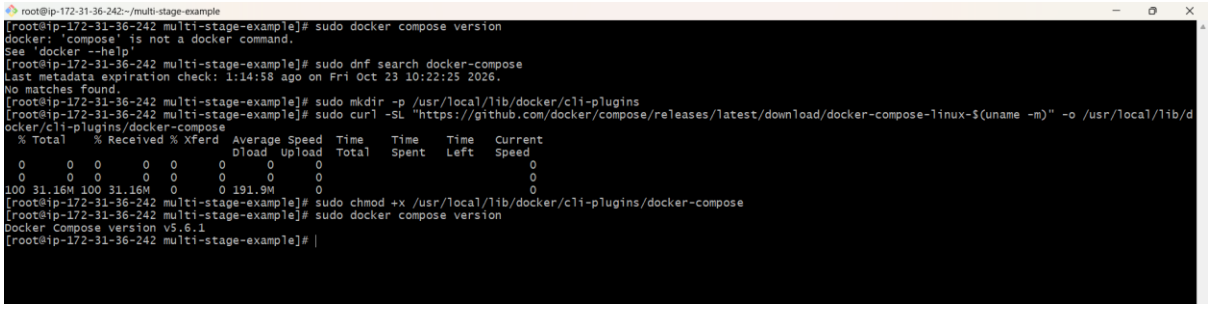
Step 7: Access Nginx through a browser.

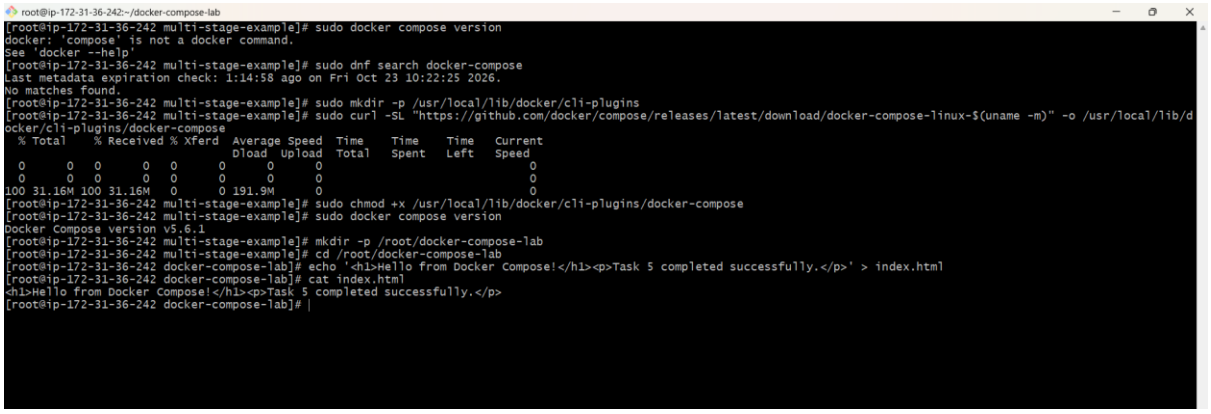
http://EC2_PUBLIC_IP:8086

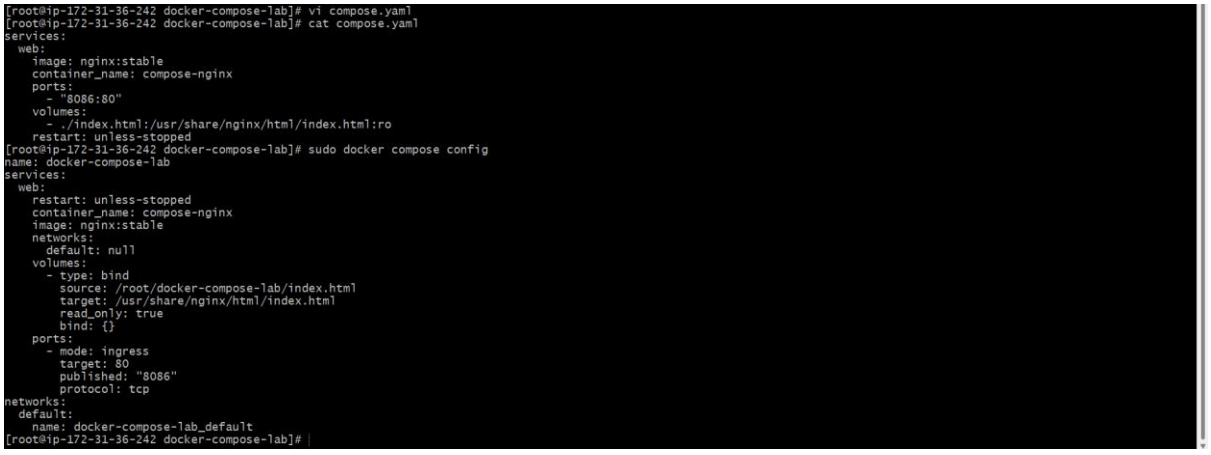
Ensure the EC2 security group permits inbound TCP traffic on port 8086.

Step 8: Stop the application when required.

sudo docker compose down











6. Implement solution to scan images when pushed to docker registry.

Implementation Steps

Step 1: Sign in to Docker Hub from the terminal.

sudo docker login

Authenticate using the Docker Hub username and an appropriate access token.

Step 2: Tag the Docker image.

If my-nginx-image:v2 is available locally:

sudo docker tag my-nginx-image:v2 poojith7878/my-nginx-image:scout-test

Step 3: Push the image to Docker Hub.

sudo docker push poojith7878/my-nginx-image:scout-test

Step 4: Open the Docker Hub repository.

https://hub.docker.com/repositories/poojith7878

Open the my-nginx-image repository and select the scout-test tag.

Step 5: View the Docker Scout analysis.

Open the Scout or vulnerability analysis section for the image.

If Docker Scout CLI is available, run:

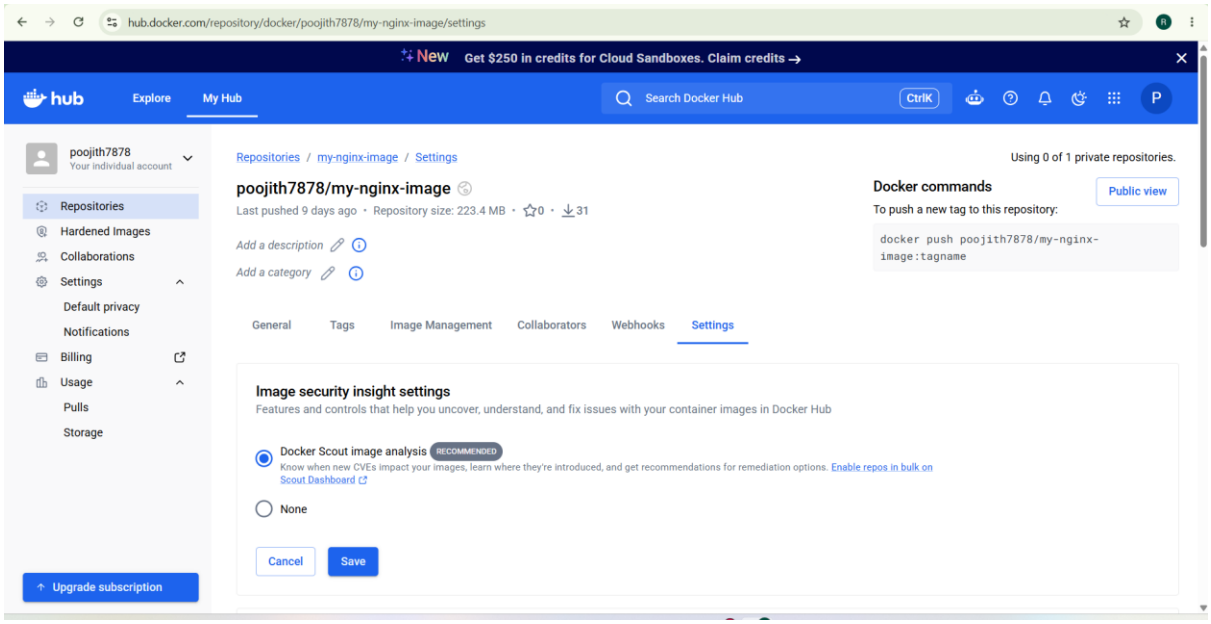
sudo docker scout quickview poojith7878/my-nginx-image:scout-test

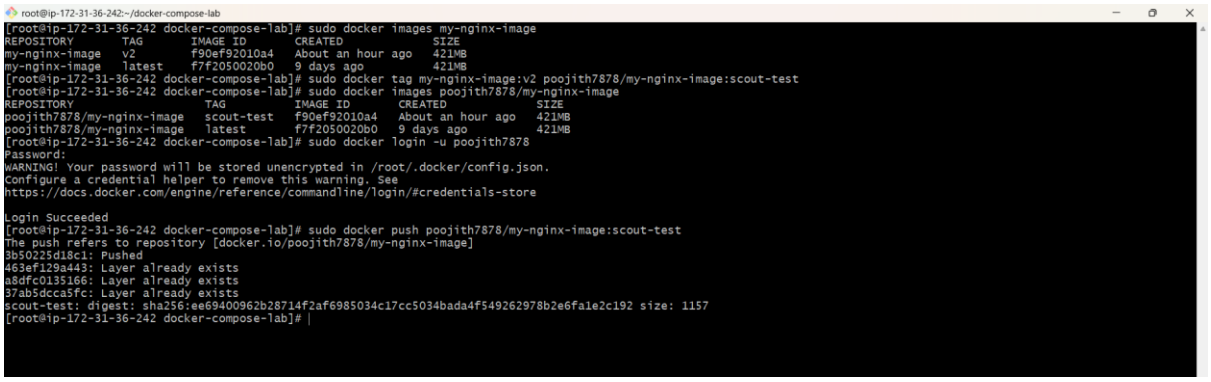
To view vulnerability details:

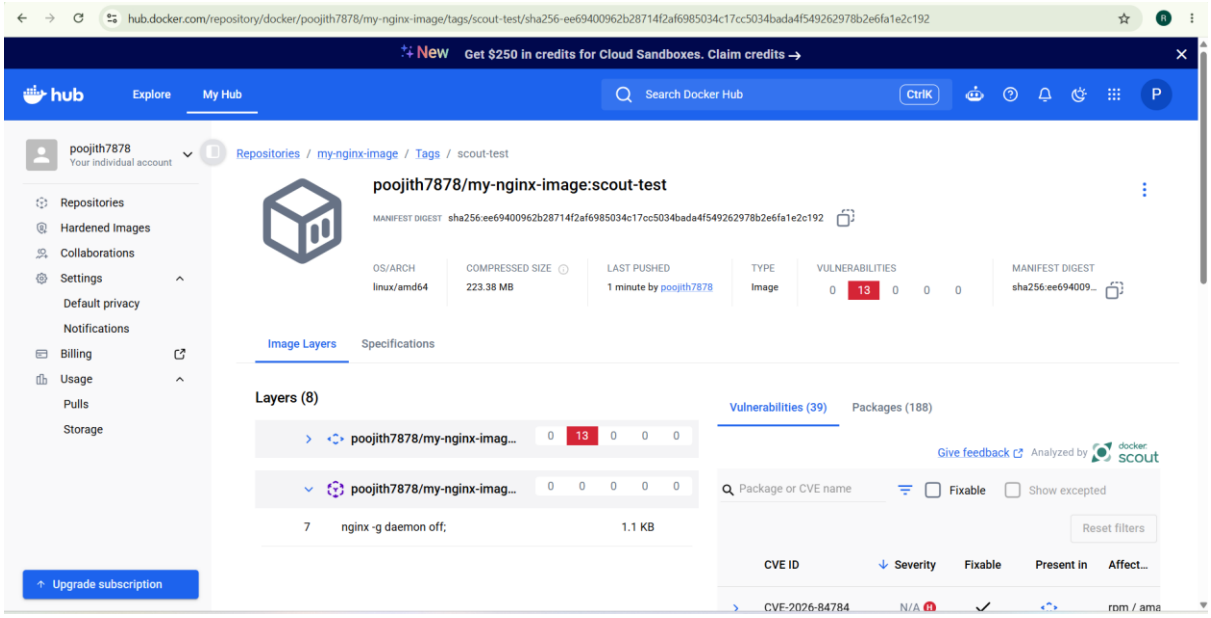
sudo docker scout cves poojith7878/my-nginx-image:scout-test

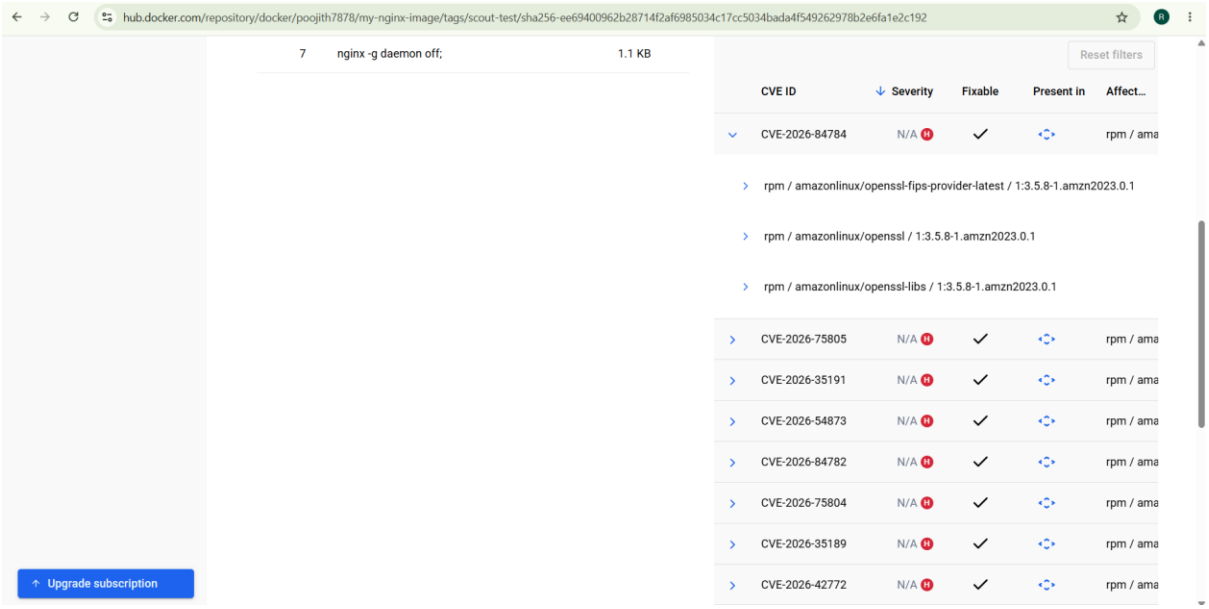
Step 6: Record the findings.

Record the vulnerability counts displayed in the analysis report, including Critical, High, Medium, and Low severity findings.


hub.docker.com/repository/docker/poojith7878/my-nginx-image/settings
New Get $250 in credits for Cloud Sandboxes. Claim credits →
hub
Explore
My Hub
Search Docker Hub
CtrlK
poojith7878
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade subscription
Repositories / my-nginx-image / Settings
Using 0 of 1 private repositories.
poojith7878/my-nginx-image
Last pushed 9 days ago • Repository size: 223.4 MB • 0 • 31
Add a description
Add a category
Docker commands
Public view
To push a new tag to this repository:
docker push poojith7878/my-nginx-image:tagname
General
Tags
Image Management
Collaborators
Webhooks
Settings
Image security insight settings
Features and controls that help you uncover, understand, and fix issues with your container images in Docker Hub
Docker Scout image analysis
RECOMMENDED
Know when new CVEs impact your images, learn where they're introduced, and get recommendations for remediation options. Enable repos in bulk on Scout Dashboard
None
Cancel
Save


root@ip-172-31-36-242:~/docker-compose-lab
[root@ip-172-31-36-242 docker-compose-lab]# sudo docker images my-nginx-image
REPOSITORY TAG IMAGE ID CREATED SIZE
my-nginx-image v2 f90ef92010a4 About an hour ago 421MB
my-nginx-image latest f7f2050020b0 9 days ago 421MB
[root@ip-172-31-36-242 docker-compose-lab]# sudo docker tag my-nginx-image:v2 poojith7878/my-nginx-image:scout-test
[root@ip-172-31-36-242 docker-compose-lab]# sudo docker images poojith7878/my-nginx-image
REPOSITORY TAG IMAGE ID CREATED SIZE
poojith7878/my-nginx-image scout-test f90ef92010a4 About an hour ago 421MB
poojith7878/my-nginx-image latest f7f2050020b0 9 days ago 421MB
[root@ip-172-31-36-242 docker-compose-lab]# sudo docker login -u poojith7878
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store
Login Succeeded
[root@ip-172-31-36-242 docker-compose-lab]# sudo docker push poojith7878/my-nginx-image:scout-test
The push refers to repository [docker.io/poojith7878/my-nginx-image]
3b50225d18c1: Pushed
463ef129a443: Layer already exists
a8dfc0135166: Layer already exists
37ab5dcca5fc: Layer already exists
scout-test: digest: sha256:ee69400962b28714f2af6985034c17cc5034bada4f549262978b2e6fa1e2c192 size: 1157
[root@ip-172-31-36-242 docker-compose-lab]#


hub.docker.com/repository/docker/poojith7878/my-nginx-image/tags/scout-test/sha256-ee69400962b28714f2af6985034c17cc5034bada4f549262978b2e6fa1e2c192
New Get $250 in credits for Cloud Sandboxes. Claim credits →
hub
Explore
My Hub
Search Docker Hub
CtrlK
poojith7878
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade subscription
Repositories / my-nginx-image / Tags / scout-test
poojith7878/my-nginx-image:scout-test
MANIFEST DIGEST sha256:ee69400962b28714f2af6985034c17cc5034bada4f549262978b2e6fa1e2c192
OS/ARCH linux/amd64
COMPRESSED SIZE 223.38 MB
LAST PUSHED 1 minute by poojith7878
TYPE Image
VULNERABILITIES 0 13 0 0 0
MANIFEST DIGEST sha256:ee694009...
Image Layers
Specifications
Layers (8)
poojith7878/my-nginx-imag... 0 13 0 0 0
poojith7878/my-nginx-imag... 0 0 0 0 0
7 nginx -g daemon off; 1.1 KB
Vulnerabilities (39)
Packages (188)
Give feedback
Analyzed by docker scout
Package or CVE name
Fixable
Show excepted
Reset filters
CVE ID
Severity
Fixable
Present in
Affect...
CVE-2026-84784
N/A
rpm / ama



7. Implement solution to scan images when pushed to AWS ECR.

Implementation Steps

Step 1: Open Amazon ECR.

1. Sign in to the AWS Management Console.
2. Select the us-east-1 region.
3. Open Amazon Elastic Container Registry.
4. Open the repository named my-nginx-image.

Repository URI:

956035085469.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image

Step 2: Configure image scanning.

Configure the applicable ECR scanning rule for my-nginx-image and enable scan-on-push using Basic scanning.

Step 3: Authenticate Docker to ECR.

Run the following command on the EC2 instance with appropriate AWS permissions:

aws ecr get-login-password --region us-east-1 | sudo docker login --username AWS --password-stdin 956035085469.dkr.ecr.us-east-1.amazonaws.com

Step 4: Tag the image for ECR.

sudo docker tag poojith7878/my-nginx-image:scout-test 956035085469.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image:scout-test

Step 5: Push the image to ECR.

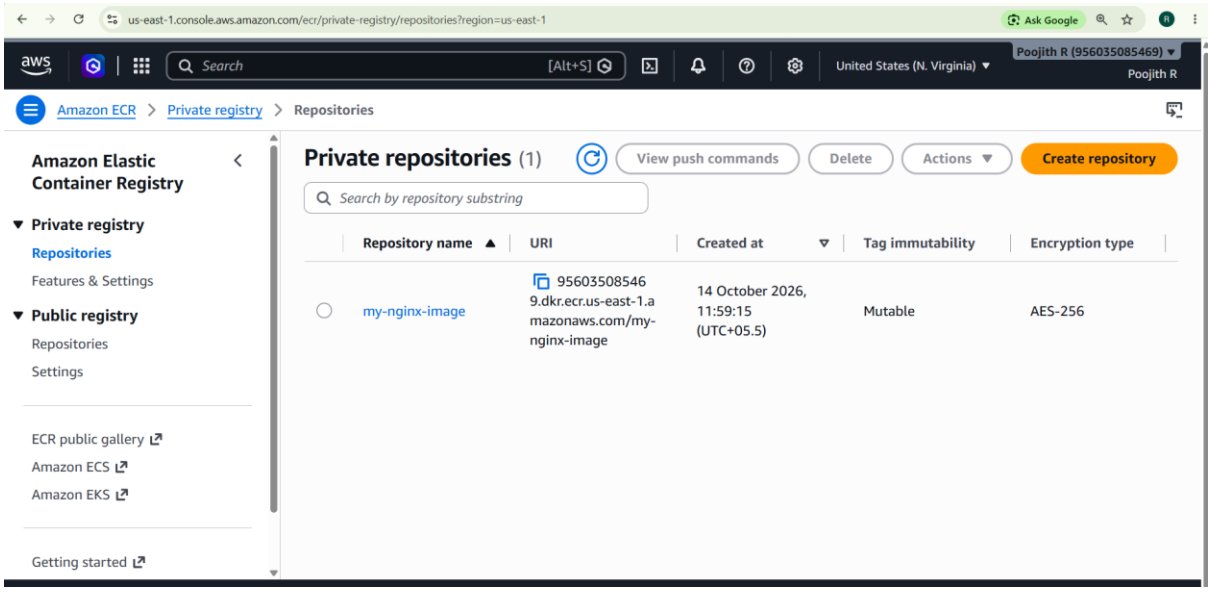
sudo docker push 956035085469.dkr.ecr.us-east-1.amazonaws.com/my-nginx-image:scout-test

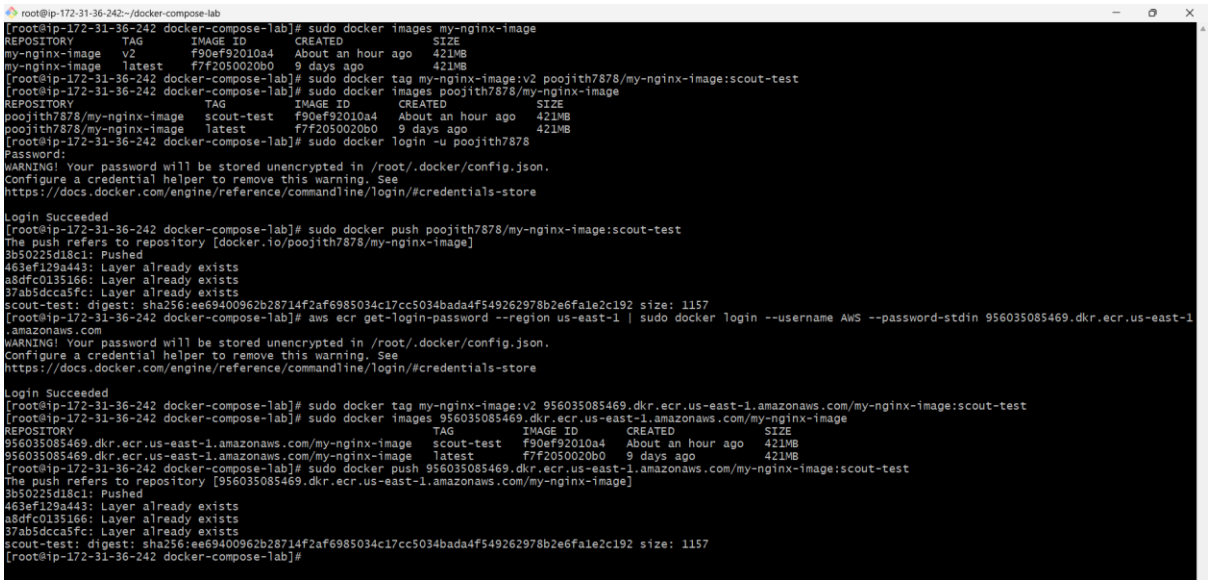
Step 6: View the image scan results.

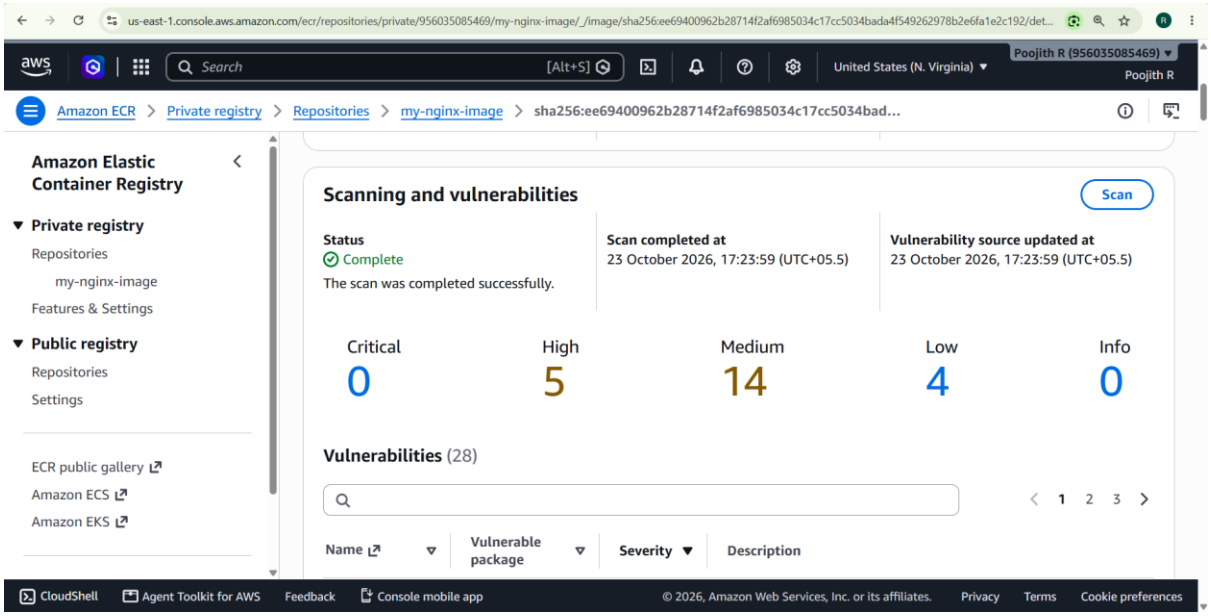
1. Open the ECR repository.
2. Select the scout-test image.
3. Open the image scanning findings.
4. Record the severity-wise findings displayed in the AWS Console.

The practical recorded the following individual severity counts:

Severity	Findings
Critical	0
High	5
Medium	14
Low	4
Info	0







8. Create a Jenkins pipeline to create a docker image and push the image to Docker hub.

Implementation Steps

Step 1: Verify Jenkins and Docker services.

sudo systemctl status jenkins

sudo systemctl status docker

Step 2: Configure Docker access for Jenkins.

sudo usermod -aG docker jenkins

Restart Jenkins:

sudo systemctl restart jenkins

Verify Docker access:

sudo -u jenkins docker ps

Step 3: Prepare the GitHub repository.

Repository URL:

https://github.com/ravutla-10/jenkins-docker-pipeline

The repository contains:

- Dockerfile
- index.html

Step 4: Configure Docker Hub credentials in Jenkins.

1. Open Jenkins Dashboard.
2. Navigate to **Manage Jenkins → Credentials**.
3. Add a credential with the following details:

Field	**Value**
Kind	Username with password
Username	poojith7878
Password	Docker Hub access token with Read & Write permissions
ID	dockerhub-creds

Save the credential.

Step 5: Create the Pipeline job.

1. Click **New Item**.
2. Enter DockerHub-Pipeline.
3. Select **Pipeline**.
4. Click **OK**.
5. Scroll to the Pipeline section.
6. Set Definition to **Pipeline script**.

Step 6: Configure the pipeline script.

Paste the following script into the Pipeline script field:

```
pipeline {
  agent any

  environment {
    DOCKER_IMAGE = 'poojith7878/my-nginx-image'
  }

  stages {
    stage('Clone GitHub Repository') {
      steps {
        git branch: 'main',
          url: 'https://github.com/ravutla-10/jenkins-docker-pipeline.git'
      }
    }

    stage('Build Docker Image') {
      steps {
        sh '''
          docker build -t ${DOCKER_IMAGE}:${BUILD_NUMBER} .
          docker tag ${DOCKER_IMAGE}:${BUILD_NUMBER} ${DOCKER_IMAGE}:latest
        '''
      }
    }

    stage('Login to Docker Hub') {
      steps {
        withCredentials([usernamePassword(
```

```
            credentialsId: 'dockerhub-creds',
            usernameVariable: 'DOCKERHUB_USER',
            passwordVariable: 'DOCKERHUB_TOKEN'
        )]) {
            sh '''
                echo "$DOCKERHUB_TOKEN" | docker login \
                    -u "$DOCKERHUB_USER" --password-stdin
            '''
        }
      }
    }

    stage('Push Docker Image') {
      steps {
        sh '''
            docker push ${DOCKER_IMAGE}:${BUILD_NUMBER}
            docker push ${DOCKER_IMAGE}:latest
        '''
      }
    }
}

post {
    success {
      echo 'Docker image built and pushed successfully!'
    }
    failure {
      echo 'Pipeline failed. Check the Console Output.'
```

```
    }
    always {
      sh 'docker logout || true'
    }
  }
}
```

Step 7: Save the job.

Click **Save**.

Step 8: Execute the pipeline.

1. Open DockerHub-Pipeline.
2. Click **Build Now**.
3. Open the build number.
4. Click **Console Output**.
5. Verify that the pipeline stages complete successfully.

Step 9: Verify the published images.

Open the Docker Hub repository:

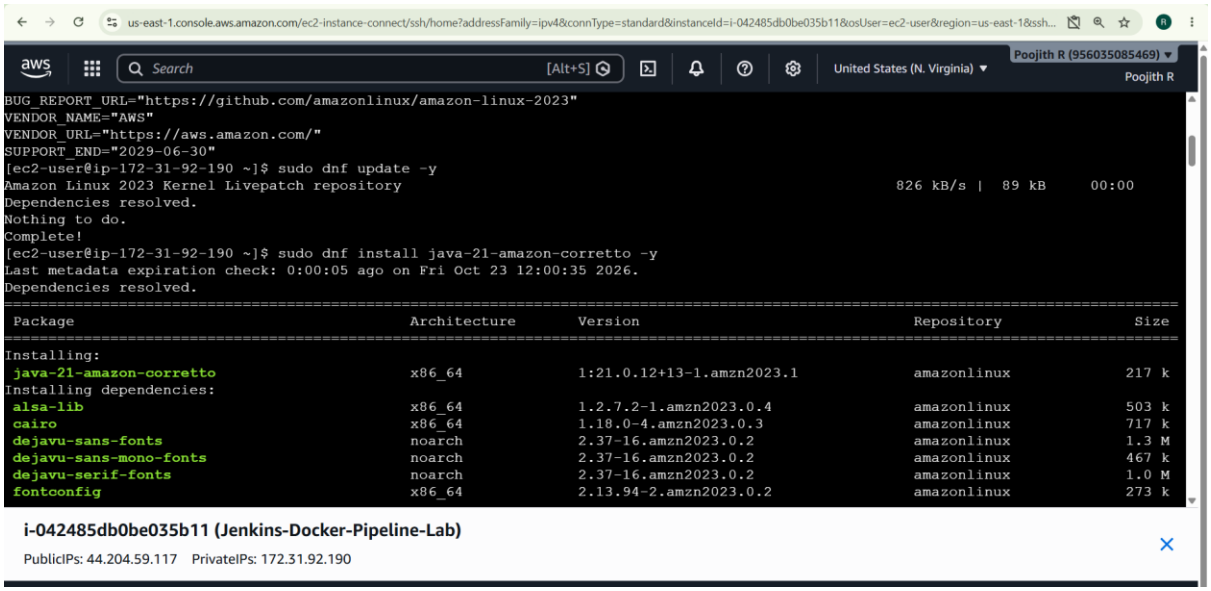
https://hub.docker.com/repositories/poojith7878

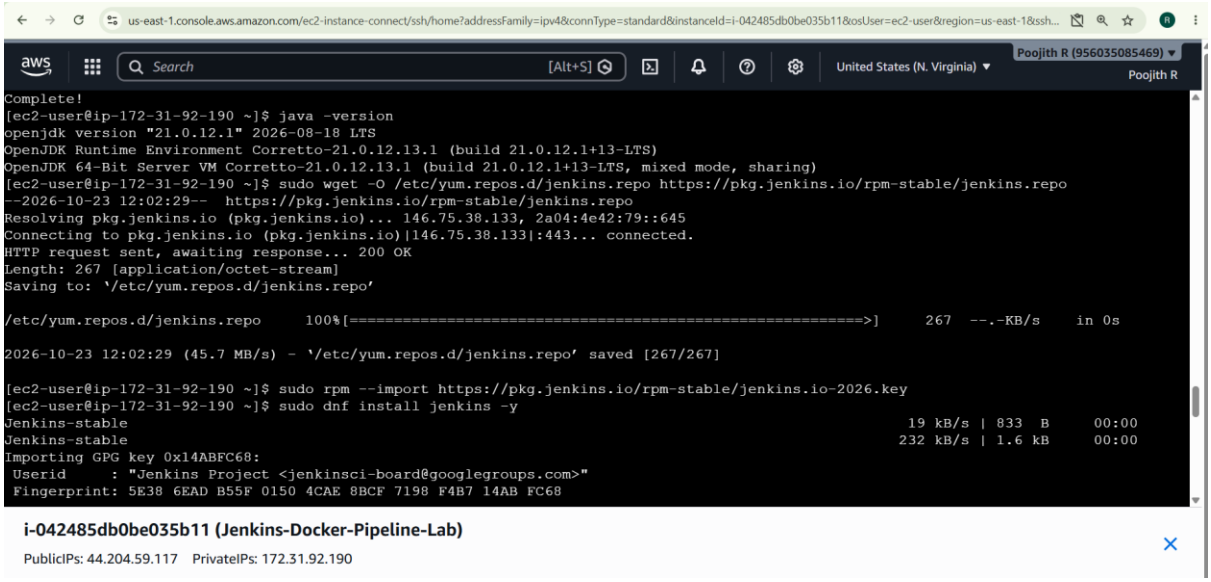
Verify these image tags:

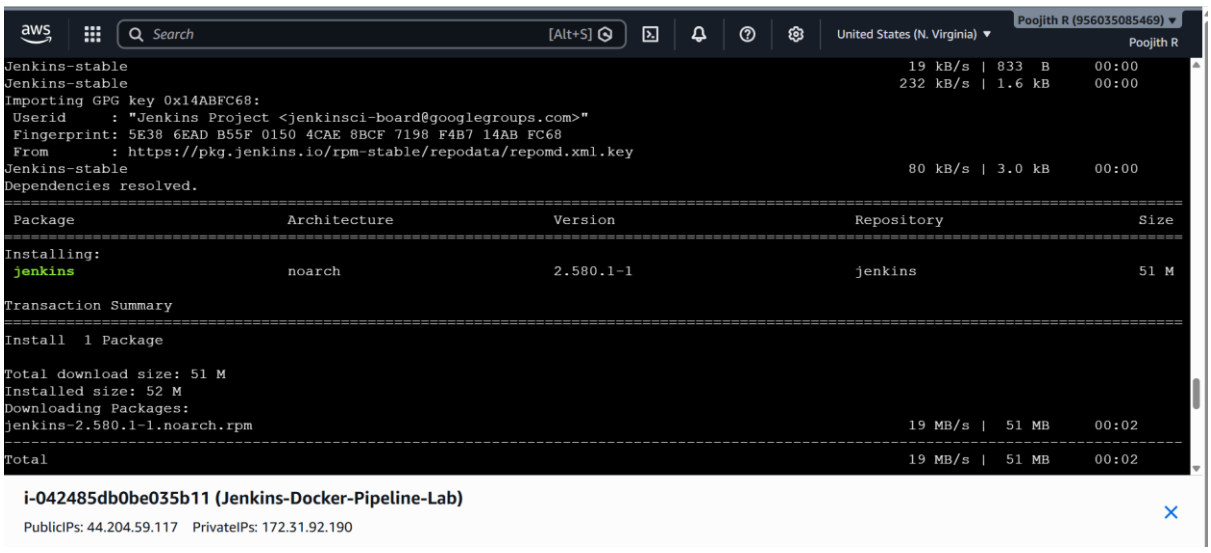
poojith7878/my-nginx-image:2

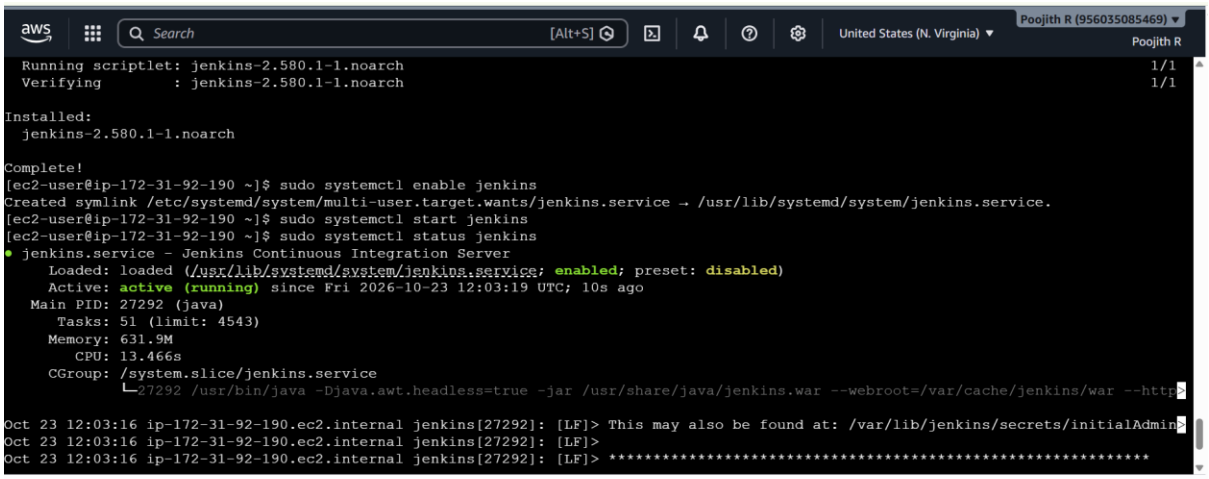
poojith7878/my-nginx-image:latest

The completed execution displayed Finished: SUCCESS.

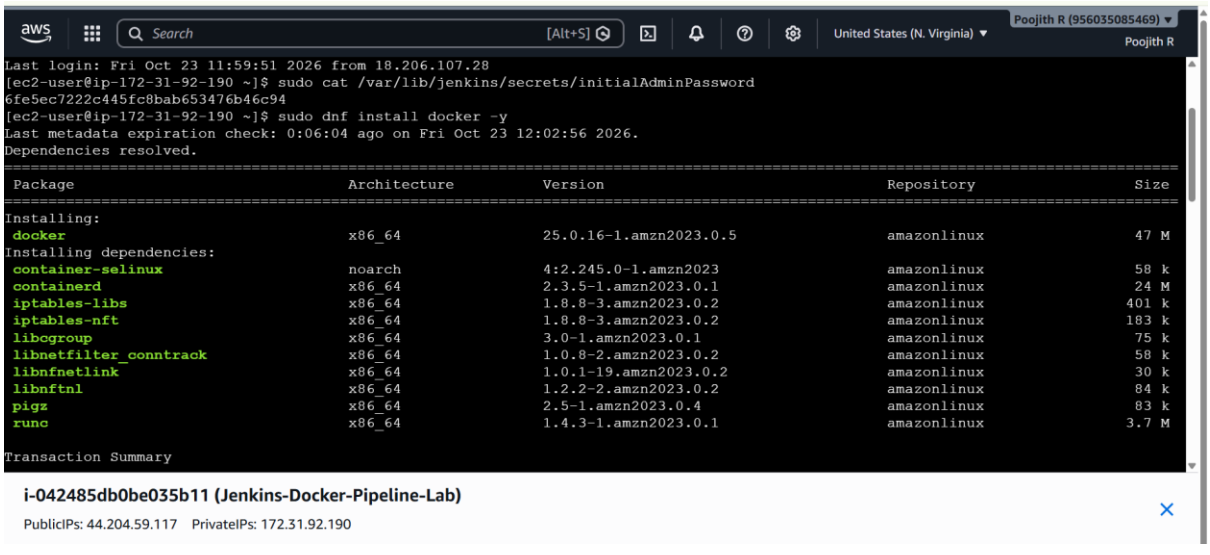

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-042485db0be035b11&osUser=ec2-user&region=us-east-1&ssh...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
BUG_REPORT_URL="https://github.com/amazonlinux/amazon-linux-2023"
VENDOR_NAME="AWS"
VENDOR_URL="https://aws.amazon.com/"
SUPPORT_END="2029-06-30"
[ec2-user@ip-172-31-92-190 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository 826 kB/s | 89 kB 00:00
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-92-190 ~]$ sudo dnf install java-21-amazon-corretto -y
Last metadata expiration check: 0:00:05 ago on Fri Oct 23 12:00:35 2026.
Dependencies resolved.
Package Architecture Version Repository Size
Installing:
java-21-amazon-corretto x86_64 1:21.0.12+13-1.amzn2023.1 amazonlinux 217 k
Installing dependencies:
alsa-lib x86_64 1.2.7.2-1.amzn2023.0.4 amazonlinux 503 k
cairo x86_64 1.18.0-4.amzn2023.0.3 amazonlinux 717 k
dejavu-sans-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 1.3 M
dejavu-sans-mono-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 467 k
dejavu-serif-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 1.0 M
fontconfig x86_64 2.13.94-2.amzn2023.0.2 amazonlinux 273 k
i-042485db0be035b11 (Jenkins-Docker-Pipeline-Lab)
PublicIPs: 44.204.59.117 PrivateIPs: 172.31.92.190

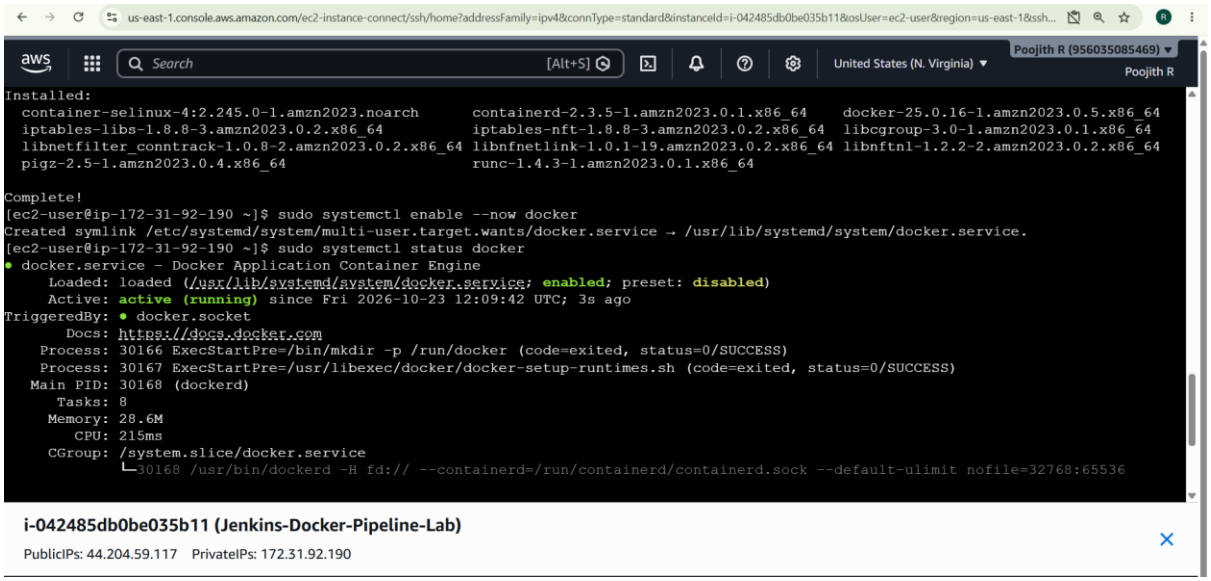

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-042485db0be035b11&osUser=ec2-user&region=us-east-1&ssh...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Complete!
[ec2-user@ip-172-31-92-190 ~]$ java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.13.1 (build 21.0.12.1+13-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.13.1 (build 21.0.12.1+13-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-92-190 ~]$ sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/rpm-stable/jenkins.repo
--2026-10-23 12:02:29-- https://pkg.jenkins.io/rpm-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.38.133, 2a04:4e42:79::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|146.75.38.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 267 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'
/etc/yum.repos.d/jenkins.repo 100%[=>] 267 --.-KB/s in 0s
2026-10-23 12:02:29 (45.7 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [267/267]
[ec2-user@ip-172-31-92-190 ~]$ sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
[ec2-user@ip-172-31-92-190 ~]$ sudo dnf install jenkins -y
Jenkins-stable 19 kB/s | 833 B 00:00
Jenkins-stable 232 kB/s | 1.6 kB 00:00
Importing GPG key 0x14ABFC68:
Userid : "Jenkins Project <jenkinsci-board@googlegroups.com>"
Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
i-042485db0be035b11 (Jenkins-Docker-Pipeline-Lab)
PublicIPs: 44.204.59.117 PrivateIPs: 172.31.92.190


aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Jenkins-stable 19 kB/s | 833 B 00:00
Jenkins-stable 232 kB/s | 1.6 kB 00:00
Importing GPG key 0x14ABFC68:
Userid : "Jenkins Project <jenkinsci-board@googlegroups.com>"
Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
From : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Jenkins-stable 80 kB/s | 3.0 kB 00:00
Dependencies resolved.
Package Architecture Version Repository Size
Installing:
jenkins noarch 2.580.1-1 jenkins 51 M
Transaction Summary
Install 1 Package
Total download size: 51 M
Installed size: 52 M
Downloading Packages:
jenkins-2.580.1-1.noarch.rpm 19 MB/s | 51 MB 00:02
Total 19 MB/s | 51 MB 00:02
i-042485db0be035b11 (Jenkins-Docker-Pipeline-Lab)
PublicIPs: 44.204.59.117 PrivateIPs: 172.31.92.190

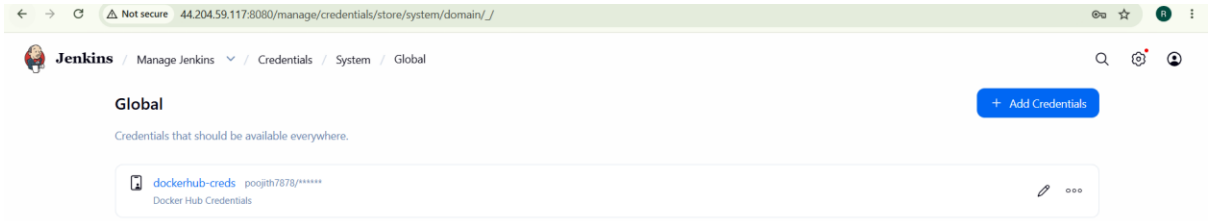

Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Running scriptlet: jenkins-2.580.1-1.noarch 1/1
Verifying : jenkins-2.580.1-1.noarch 1/1
Installed:
jenkins-2.580.1-1.noarch
Complete!
[ec2-user@ip-172-31-92-190 ~]$ sudo systemctl enable jenkins
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service → /usr/lib/systemd/system/jenkins.service.
[ec2-user@ip-172-31-92-190 ~]$ sudo systemctl start jenkins
[ec2-user@ip-172-31-92-190 ~]$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
Active: active (running) since Fri 2026-10-23 12:03:19 UTC; 10s ago
Main PID: 27292 (java)
Tasks: 51 (limit: 4543)
Memory: 631.9M
CPU: 13.466s
CGroup: /system.slice/jenkins.service
└27292 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --http
Oct 23 12:03:16 ip-172-31-92-190.ec2.internal jenkins[27292]: [LF]> This may also be found at: /var/lib/jenkins/secrets/initialAdmin
Oct 23 12:03:16 ip-172-31-92-190.ec2.internal jenkins[27292]: [LF]>
Oct 23 12:03:16 ip-172-31-92-190.ec2.internal jenkins[27292]: [LF]> **********
i-042485db0be035b11 (Jenkins-Docker-Pipeline-Lab)
PublicIPs: 44.204.59.117 PrivateIPs: 172.31.92.190


Not secure
44.204.59.117:8080/login?from=%2F
Getting Started
Unlock Jenkins
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:
/var/lib/jenkins/secrets/initialAdminPassword
Please copy the password from either location and paste it below.
Administrator password
Continue


Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Last login: Fri Oct 23 11:59:51 2026 from 18.206.107.28
[ec2-user@ip-172-31-92-190 ~]$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
6fe5ec7222c445fc8bab653476b46c94
[ec2-user@ip-172-31-92-190 ~]$ sudo dnf install docker -y
Last metadata expiration check: 0:06:04 ago on Fri Oct 23 12:02:56 2026.
Dependencies resolved.
Package Architecture Version Repository Size
Installing:
docker x86_64 25.0.16-1.amzn2023.0.5 amazonlinux 47 M
Installing dependencies:
container-selinux noarch 4:2.245.0-1.amzn2023 amazonlinux 58 k
containerd x86_64 2.3.5-1.amzn2023.0.1 amazonlinux 24 M
iptables-libs x86_64 1.8.8-3.amzn2023.0.2 amazonlinux 401 k
iptables-nft x86_64 1.8.8-3.amzn2023.0.2 amazonlinux 183 k
libcgroup x86_64 3.0-1.amzn2023.0.1 amazonlinux 75 k
libnetfilter_conntrack x86_64 1.0.8-2.amzn2023.0.2 amazonlinux 58 k
libnfnetlink x86_64 1.0.1-19.amzn2023.0.2 amazonlinux 30 k
libnftnl x86_64 1.2.2-2.amzn2023.0.2 amazonlinux 84 k
pigz x86_64 2.5-1.amzn2023.0.4 amazonlinux 83 k
runc x86_64 1.4.3-1.amzn2023.0.1 amazonlinux 3.7 M
Transaction Summary
i-042485db0be035b11 (Jenkins-Docker-Pipeline-Lab)
PublicIPs: 44.204.59.117 PrivateIPs: 172.31.92.190

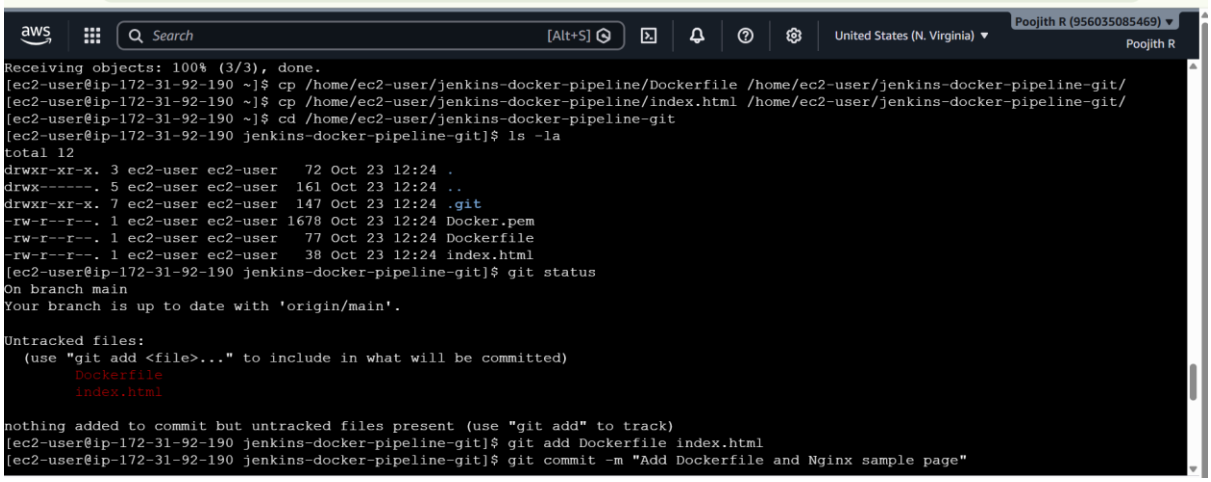



```
[ec2-user@ip-172-31-92-190 ~]$ sudo usermod -aG docker jenkins
[ec2-user@ip-172-31-92-190 ~]$ sudo systemctl restart jenkins
sudo -u jenkins docker ps
[ec2-user@ip-172-31-92-190 ~]$ sudo -u jenkins docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[ec2-user@ip-172-31-92-190 ~]$
```

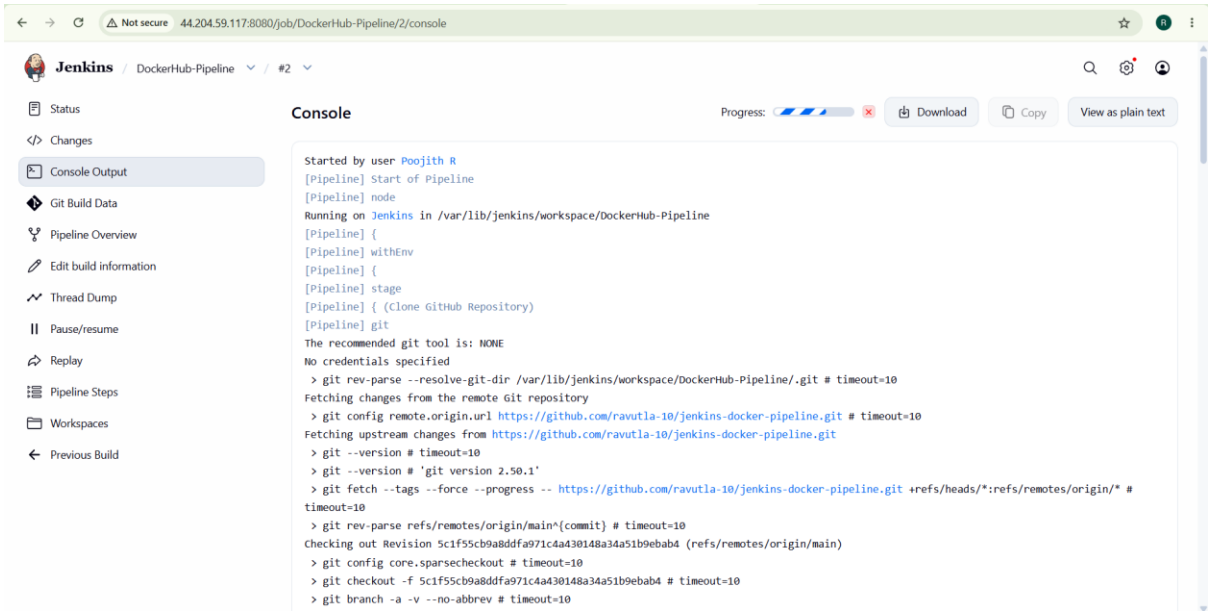


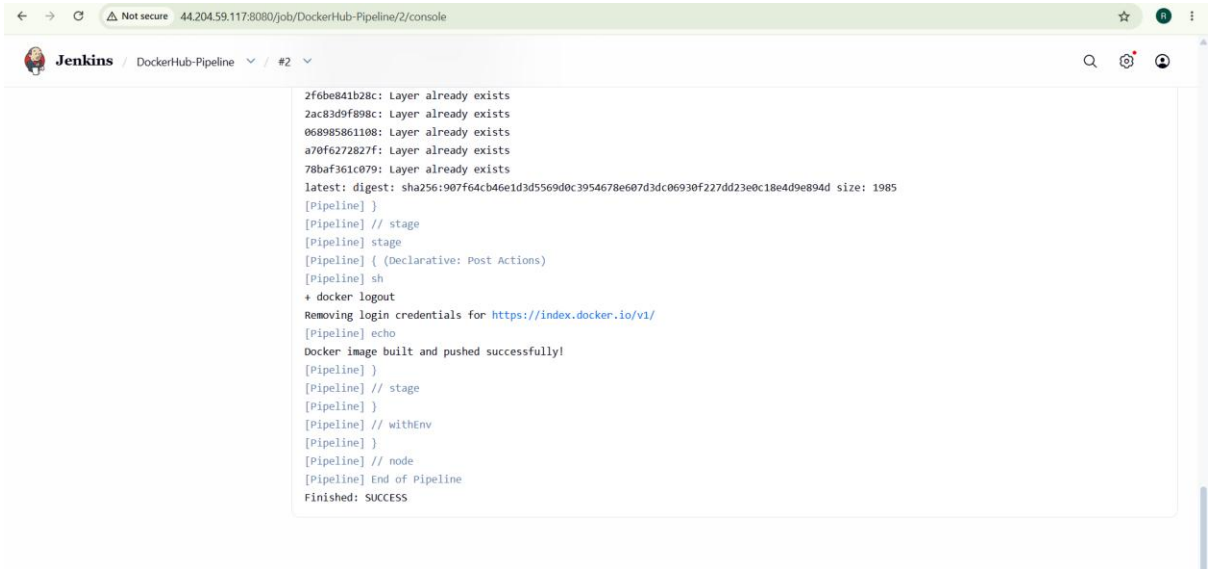

```
      _/m/'
Last login: Fri Oct 23 12:06:40 2026 from 18.206.107.29
[ec2-user@ip-172-31-92-190 ~]$ mkdir -p ~/jenkins-docker-pipeline
cd ~/jenkins-docker-pipeline
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline]$ echo '<h1>Hello from Jenkins Pipeline!</h1>' > index.html
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline]$ cat > Dockerfile <<'EOF'
FROM nginx:stable
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
EOF
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline]$ ls -l
total 8
-rw-r--r--. 1 ec2-user ec2-user 77 Oct 23 12:17 Dockerfile
-rw-r--r--. 1 ec2-user ec2-user 38 Oct 23 12:17 index.html
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline]$ git --version
-bash: git: command not found
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline]$ sudo yum install git
Last metadata expiration check: 0:16:57 ago on Fri Oct 23 12:02:56 2026.
Dependencies resolved.
================================================================================
```

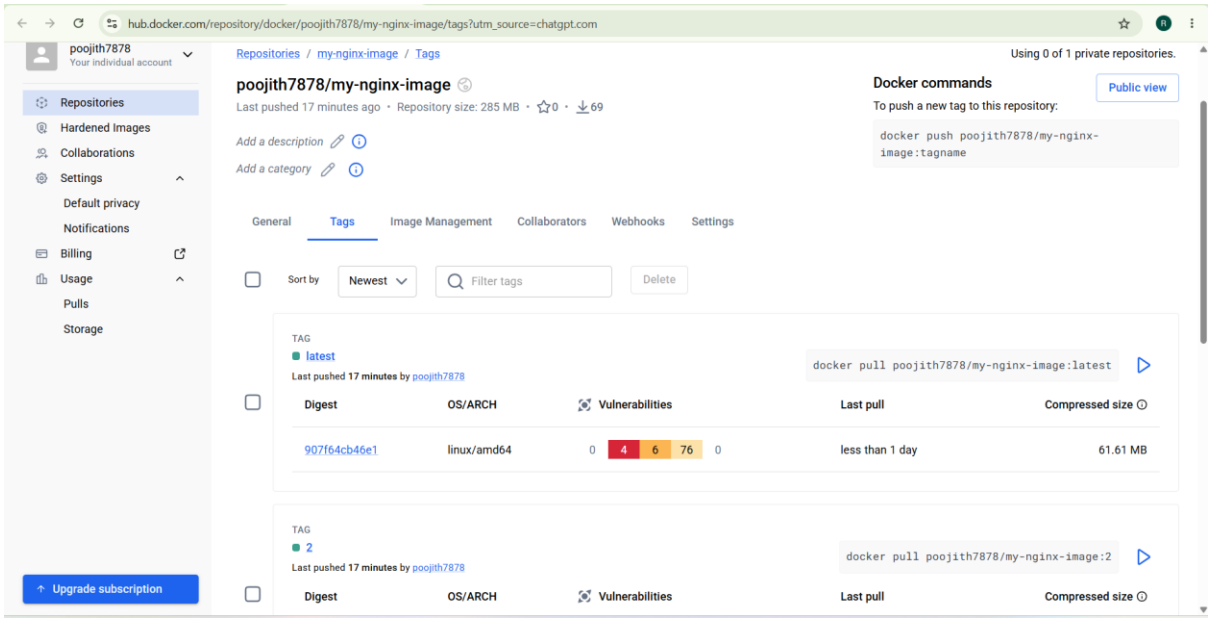
```
fa39ac470842296231b7d3b7077da864b2bf4e85        refs/heads/main
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline]$ pwd
/home/ec2-user/jenkins-docker-pipeline
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline]$ ls -la
total 8
drwxr-xr-x. 2 ec2-user ec2-user  42 Oct 23 12:20 .
drwx------. 4 ec2-user ec2-user 126 Oct 23 12:17 ..
-rw-r--r--. 1 ec2-user ec2-user  77 Oct 23 12:17 Dockerfile
-rw-r--r--. 1 ec2-user ec2-user  38 Oct 23 12:17 index.html
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline]$ git status
fatal: not a git repository (or any of the parent directories): .git
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline]$ cd /home/ec2-user
[ec2-user@ip-172-31-92-190 ~]$ git clone https://github.com/ravutla-10/jenkins-docker-pipeline.git jenkins-docker-pipeline-git
Cloning into 'jenkins-docker-pipeline-git'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
[ec2-user@ip-172-31-92-190 ~]$ cp /home/ec2-user/jenkins-docker-pipeline/Dockerfile /home/ec2-user/jenkins-docker-pipeline-git/
[ec2-user@ip-172-31-92-190 ~]$ cp /home/ec2-user/jenkins-docker-pipeline/index.html /home/ec2-user/jenkins-docker-pipeline-git/
[ec2-user@ip-172-31-92-190 ~]$ cd /home/ec2-user/jenkins-docker-pipeline-git
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline-git]$ ls -la
total 12
```




```
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline-git]$ git push origin main
Username for 'https://github.com': ravutla-10
Password for 'https://ravutla-10@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 459 bytes | 459.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ravutla-10/jenkins-docker-pipeline.git
   fa39ac4..5c1f55c  main -> main
[ec2-user@ip-172-31-92-190 jenkins-docker-pipeline-git]$
```


Not secure 44.204.59.117:8080/job/DockerHub-Pipeline/2/console
Jenkins / DockerHub-Pipeline / #2
Status
Changes
Console Output
Git Build Data
Pipeline Overview
Edit build information
Thread Dump
Pause/resume
Replay
Pipeline Steps
Workspaces
Previous Build
Console
Progress:
Download
Copy
View as plain text
Started by user Poojith R
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/DockerHub-Pipeline
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Clone GitHub Repository)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/DockerHub-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/ravutla-10/jenkins-docker-pipeline.git # timeout=10
Fetching upstream changes from https://github.com/ravutla-10/jenkins-docker-pipeline.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/ravutla-10/jenkins-docker-pipeline.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5c1f55cb9a8ddfa971c4a430148a34a51b9ebab4 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 5c1f55cb9a8ddfa971c4a430148a34a51b9ebab4 # timeout=10
> git branch -a -v --no-abbrev # timeout=10


Not secure 44.204.59.117:8080/job/DockerHub-Pipeline/2/console
Jenkins / DockerHub-Pipeline / #2
2f6be841b28c: Layer already exists
2ac83d9f898c: Layer already exists
068985861108: Layer already exists
a70f6272827f: Layer already exists
78baf361c079: Layer already exists
latest: digest: sha256:907f64cb46e1d3d5569d0c3954678e607d3dc06930f227dd23e0c18e4d9e894d size: 1985
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
+ docker logout
Removing login credentials for https://index.docker.io/v1/
[Pipeline] echo
Docker image built and pushed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


hub.docker.com/repository/docker/poojith7878/my-nginx-image/tags?utm_source=chatgpt.com
poojith7878
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade subscription
Repositories / my-nginx-image / Tags
Using 0 of 1 private repositories.
poojith7878/my-nginx-image
Last pushed 17 minutes ago • Repository size: 285 MB • 0 • 69
Add a description
Add a category
Docker commands
Public view
To push a new tag to this repository:
docker push poojith7878/my-nginx-image:tagname
General
Tags
Image Management
Collaborators
Webhooks
Settings
Sort by
Newest
Filter tags
Delete
TAG
latest
Last pushed 17 minutes by poojith7878
docker pull poojith7878/my-nginx-image:latest
Digest
OS/ARCH
Vulnerabilities
Last pull
Compressed size
907f64cb46e1
linux/amd64
0 4 6 76 0
less than 1 day
61.61 MB
TAG
2
Last pushed 17 minutes by poojith7878
docker pull poojith7878/my-nginx-image:2
Digest
OS/ARCH
Vulnerabilities
Last pull
Compressed size